

Linnest Typst APIs

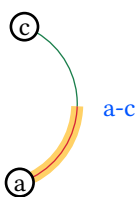
Linnest exposes the `linnest.wasm` graph layout plugin through a small Typst API. Its drawing layer imports the sibling Kurvst package; the standalone Kurvst manual documents the full curve and path-pattern API. The public surface is intentionally narrow:

- `graph` for construction, parsing, inspection, joins, and graph algorithms.
- `subgraph` for subgraph object construction and inspection.
- `layout` for the separate layout pass.
- `draw` for rendering a laid-out graph object with CeTZ.
- `physics` for reusable particle-line edge styles.

Minimal Builder Example

```
#import "../src/lib.typ": draw, graph, layout, subgraph

#let b = graph.builder(name: "demo")
#let (node: a, builder: b) = graph.node(b, name: "a")
#let (node: c, builder: b) = graph.node(b, name: "c")
#let b = graph.edge(
  b,
  source: (node: a, compass: "e"),
  sink: (node: c, compass: "w"),
  statements: (
    color: "0055ff",
    source-color: "d72638",
    sink-color: "1b7f4c",
    label: "a-c",
  ),
),
)
#let g = graph.finish(b)
#let g = layout(g)
#let east = subgraph.compass(g, "e")
#let edges = graph.edges(g, subgraph: east)
#let dot = graph.dot(g)
#let edge-label(edge) = text(fill: rgb("#" + edge.color))[#edge.label]
#let source-style(edge) = (stroke: rgb("#" + edge.source-color) + 0.5pt)
#let sink-style(edge) = (stroke: rgb("#" + edge.sink-color) + 0.5pt)
#draw(g, subgraph: east, edge-label: edge-label, source-style: source-style, sink-style: sink-style)
```



Graph Objects

Graph, builder, and subgraph objects are opaque zero-copy values. Build or parse graph objects with `graph`, transform graph objects with `layout`, and pass objects back to `graph` or `subgraph` for inspection.

- `graph.parse(input)` parses one or more DOT digraphs and returns an array of graph objects.
- `graph.build(...)` constructs one graph object from named arguments. This is convenience sugar over the builder functions.
- `graph.builder(...)` starts a builder object.
- `graph.node(builder, ...)` returns (node: index, builder: builder).
- `graph.edge(builder, ...)` returns the updated builder.
- `graph.finish(builder)` turns a builder into a graph.
- `layout(graph, ...)` runs layout as an explicit second step. Its settings are named parameters so calls stay descriptive and Tidy can document each field.
- `draw(graph, ...)` draws a laid-out graph object with CeTZ.

- `graph.dot(graph)` returns a DOT string for inspection or export.

Builder Spec

The builder API is the preferred construction API when nodes need to be reused. Use destructuring to keep the builder value moving:

```
#let (node: a, builder: b) = graph.node(b, name: "a")
```

`graph.edge` accepts source and sink half-edge dictionaries. A half edge can contain node, statement, id, port-label, compass, and in-subgraph. Set source: none or sink: none to create an external half edge.

String-valued DOT statements may contain {name} placeholders. `graph.build` and the builder object expand each placeholder while constructing the graph object. Use {{ and }} for literal braces. Unknown placeholders are left unchanged.

This is useful for graph-level edge-statements: the default edge statement is merged into each edge, then expanded against that edge's complete statement dictionary. The following default metadata records a display label derived from the per-edge label statement:

```
#let b = graph.builder(
  edge-statements: (
    color: "000000",
    display-label: "{label}",
  ),
)
#let (node: a, builder: b) = graph.node(b, name: "a")
#let (node: c, builder: b) = graph.node(b, name: "c")
#let b = graph.edge(
  b,
  source: (node: a),
  sink: (node: c),
  statements: (
    color: "0055ff",
    label: "a-c",
  ),
)
```

Placements

`graph.pos` creates a first-class placement. The default mode: "pin" turns a coordinate into a fixed layout constraint and a drawable position. Use mode: "start" when the coordinate should only seed the layout:

```
#let (node: a, builder: b) = graph.node(b, name: "a", pos: graph.pos(x: -2, y: 0))
#let (node: c, builder: b) = graph.node(b, name: "c", pos: graph.pos(ref: a, dx: 4, dy: 0))
#let b = graph.edge(b, source: (node: a), sink: (node: c), pos: graph.pos(x: 0, y: 1.2))
```

`graph.group` links one coordinate across several nodes or edge control points. A side of "+" keeps the coordinate positive, and "-" keeps it negative. GammaLoop external-edge columns use this to keep incoming and outgoing external legs on opposite sides while pairing rows by a shared y group:

```
#let b = graph.edge(
  b,
  source: (node: right-ext),
  sink: (node: center),
  pos: graph.pos(x: graph.group("right", side: "+"), y: graph.group("edgee0")),
)
#let b = graph.edge(
  b,
  source: (node: center),
  sink: (node: left-ext),
  pos: graph.pos(x: graph.group("left", side: "-"), y: graph.group("edgee0")),
)
```

Raw DOT input uses the same placement model through the `pos` attribute. The standard Graphviz subset is preserved: `pos="x,y"` is a starting coordinate and `pos="x,y!"` is pinned. Linnest extends the value with explicit id references and axis entries. In axis entries, ! belongs to that axis: numeric entries without ! are starting

coordinates, numeric entries with ! are fixed constraints, and grouped entries must use ! because groups are constraints.

```
digraph {
  a [id=0 pos="0,0!"]
  b [id=1 pos="ref(node:0)+4,0!"]
  a -> b [id=0 pos="ref(node:1)+0,1!"]
  b -> c [id=1 pos="ref(edge:0)+1,0!"]
  c [id=2 pos="x:2!"]
  d [id=3 pos="x:2!,y:1"]
  ext -> a [id=2 pos="x:@-left!,y:@edge0!"]
}
```

The DOT grammar is intentionally id-based: `ref(node:0)` uses a node id and `ref(edge:0)` uses an edge id. Bare names and implicit edge order are not placement references. The `@` syntax denotes grouped coordinate constraints; `@+name` and `@-name` keep the grouped coordinate on the positive or negative side respectively.

```
pos="x,y"           // start x and y
pos="x,y!"          // pin x and y
pos="x:<coord>"      // start numeric x
pos="x:<coord>!"     // pin x
pos="y:<coord>!"     // pin y
pos="x:<coord>!,y:<coord>" // pin x, start numeric y
pos="x:<coord>,y:<coord>!" // start numeric x, pin y
```

Draw styling is Typst-native. Pass dictionaries or callbacks to draw; edge callbacks receive the merged scope, edge statements, source/sink half-edge records, and the edge index:

```
#let edge-label(edge) = text(fill: rgb("#" + edge.color))[#edge.display-label]
#let source-style(edge) = (stroke: red + 0.5pt)
#let sink-style(edge) = (stroke: blue + 0.5pt)
#draw(layout(graph.finish(b)), edge-label: edge-label, source-style: source-style, sink-style: sink-style)
```

Marks can follow graph orientation as a first-class draw option. Put the same mark layer on both halves and set `mark-orientation: "edge"`; default-oriented edges mark the source half, reversed edges mark the sink half with the marker flipped, and undirected edges suppress the mark.

```
#let oriented-arrow = (
  stroke: black + 0.7pt,
  mark: (end: (symbol: ">", fill: black, anchor: "center", shorten-to: auto), scale: 0.75),
  mark-position: "center-if-dangling",
  mark-orientation: "edge",
)
#draw(layout(graph.finish(b)), source-style: oriented-arrow, sink-style: oriented-arrow)
```

Physics Edge Styles

`physics.style(..)` returns `source-style`, `sink-style`, and `edge-label` callbacks for draw. The default source/sink strokes use darker/lighter halves to encode the graph source/sink split. Fermion map entries marked with `fermion-flow` receive one particle-flow arrow on the main edge using `mark-orientation: "edge"`; paired edges follow `edge.orientation`, dangling half edges place the arrow in the middle of the visible half edge, and undirected edges omit the arrow. This is independent of momentum arrows.

Set `momentum-arrows: true` to draw centered parallel arrow decorations while the main edge remains connected to the nodes. The arrows flow from source to sink independently of the DOT `dir/orientation` value; there is one momentum marker per edge, even though the base edge is internally styled as source and sink halves. On paired edges the marker lives on the sink half; on dangling edges it lives on the existing source or sink half-edge. The decoration defaults to a plain black 0.55pt stroke and a scaled CeTZ "barbed" arrowhead, so it does not inherit the base particle stroke. The arrow length is capped by both `momentum-arrow-length` and `momentum-arrow-ratio`, so the shorter one wins. When the layout provides an edge-label position, the arrow offset is signed so the momentum marker is drawn on the same side of the edge as that label. Use `momentum-arrow-offset` to set the normal displacement. `momentum-arrow-mark: auto` uses the default barbed marker, `momentum-arrow-mark: none` draws only the momentum line, and any other value overrides the CeTZ mark style, for example "stealth" or

(end: "barbed",
scale: 1.6). Only the end mark is kept so source/sink splitting cannot create a second momentum head.

Optional labels can be built from edge metadata with show-edge-index, show-half-edge-index, show-particle, and, for explicit momentum fields, show-momentum. show-half-edge-index only emits a value for dangling half edges.

```
#import "../src/lib.typ": draw, graph, layout, physics

#let b = graph.builder(name: "physics")
#let (node: a, builder: b) = graph.node(b, name: "a")
#let (node: c, builder: b) = graph.node(b, name: "c")
#let b = graph.edge(
  b,
  source: (node: a),
  sink: (node: c),
  statements: (particle: "fermion", id: "7"),
)
#let b = graph.edge(b, source: (node: c), sink: none, statements: (particle: "fermion", id: "8"))
#let callbacks = physics.style(
  momentum-arrows: true,
  show-edge-index: true,
  show-particle: true,
)
#draw(
  layout(graph.finish(b)),
  source-style: callbacks.source-style,
  sink-style: callbacks.sink-style,
  edge-label: callbacks.edge-label,
)
```

Set edge-offset on draw, or offset in a source-style/sink-style dictionary, to draw a fitted parallel path. Style callbacks may also return an array of dictionaries; the layers are drawn in order on the same graph edge, so parallel strokes do not require duplicate graph edges.

```
#let base-style(edge) = (
  stroke: (paint: gray, thickness: 0.7pt, cap: "round"),
)
#let offset-style(edge) = (
  offset: 0.18,
  length: 1.4,
  ratio: 0.5,
  resolve-length: "min",
  stroke: (paint: rgb("#2f6f4e"), thickness: 1pt, cap: "round"),
)
#let source-style(edge) = (base-style(edge), offset-style(edge))
#let sink-style(edge) = (base-style(edge), offset-style(edge) + (mark: (end: ">")))
```

The parallel path is applied to the base edge geometry before patterns and other decorations; node outlets then trim the shifted path, so shifted paths still start and end outside fitted node circles. Add edge-length or length to center-trim the shifted path to a fixed arc length, and add edge-ratio or ratio to cap it by a fraction of the base edge length. edge-resolve-length

resolve-length decides how to combine both limits "min"/"shorter" (default), "max"/"longer",

"length"/"fixed", "ratio"/"relative", "none"/"full", or a function receiving (base-length, length, ratio).

Set offset-side: "label" on an offset layer to choose the sign of offset so the layer is drawn on the same side of the curve as the edge label.

The builder keeps source-style-eval, sink-style-eval, and label-eval as ordinary kebab-case edge metadata for downstream renderers or DOT export. They can be set as direct arguments on graph.build, graph.builder, or graph.edge, instead of being manually nested under edge-statements or per-edge statements:

```
#let b = graph.builder(
  name: "demo",
```

```

    source-style-eval: "(stroke: red + 0.5pt)",
    sink-style-eval: "(stroke: blue + 0.5pt)",
  )
#let b = graph.edge(
  b,
  source: (node: a),
  sink: (node: c),
  label-eval: "(text(fill: rgb(\"#{color}\"))[\"{label}\"])",
  statements: (color: "0055ff", label: "a-c"),
)

```

graph.build accepts the same data in one dictionary:

```

#let g = graph.build(
  name: "demo",
  statements: (full_num: "x + y"),
  node-statements: (shape: "circle"),
  edge-statements: (
    color: "000000",
    display-label: "{label}",
  ),
  nodes: ((name: "a"), (name: "b")),
  edges: ((
    source: (node: 0, compass: "e"),
    sink: (node: 1, compass: "w"),
    statements: (color: "0055ff", label: "ab"),
  )),
)

```

Graph Queries

graph.info(g) returns graph metadata. graph.nodes(g) returns node records, and graph.edges(g) returns edge records. Pass subgraph: sg to filter nodes or edges by an subgraph object.

graph.join(left, right, key: "statement") joins matching dangling half edges. The key is read from half-edge data and can be "statement", "port-label", "compass", or "id".

graph.cycles(g) returns subgraph objects for a cycle basis. graph.forests(g) returns subgraph objects for spanning forests.

Layout Model

layout starts from a traversal-tree placement, then optimizes the positions of graph nodes and edge control points. The initial tree spacing is

$$L = \lambda \sqrt{\frac{WH}{\max(n, 1)}}$$

,

with horizontal spacing $\tau_x L$ and vertical spacing $\tau_y L$. Here λ is length-scale, W is viewport-w, H is viewport-h, τ_x is tree-dx, and τ_y is tree-dy. These fields set the geometry scale for both layout modes.

For deterministic, non-iterative placement, use layout-algo: "tree" or layout-algo: "dot". "tree" places a traversal forest by levels. "dot" uses a directed layered placement for acyclic inputs, falling back to tree placement when the selected edges contain a cycle. Both modes also work on a subgraph:

```

#let g = graph.parse("digraph partial { a -> b; b -> c; c -> d; d -> a }").at(0)
#let tree = graph.forests(g).at(0)
#let g = layout(g, layout-algo: "tree", subgraph: tree)
#draw(g)

```

Only nodes touched by the selected subgraph are placed. Edge control points are then resolved from the current node positions, so edges outside the selected subgraph are drawn as straight lines unless their own position constraints say otherwise.

layout-algo: "force" and layout-algo: "anneal" also accept subgraph. For these iterative modes, nodes and edge control points outside the selected subgraph stay fixed and act as boundary points while the selected subgraph is optimized.

Set layout-nodes: "fixed" to keep every node at its current pos for this layout pass and move only edge control points. With subgraph, only edges in the selected subgraph are moved; all other edge control points keep their current positions. The fixed-node policy is temporary; the returned graph stores the resulting coordinates as pos, but it does not turn them into persistent pin constraints. This is useful after one node-placement pass when a later pass should route or relax selected edges without disturbing the node layout:

```
#let g = graph.parse("digraph partial { a [pos=\"0,0\"]; b [pos=\"4,0\"]; c [pos=\"8,0\"]; a -> b; b -> c }").at(0)
#let first = subgraph.bits(g, (true, true, false, false))
#let g = layout(g, layout-algo: "tree", layout-nodes: "fixed", subgraph: first)
#draw(g)
```

The shared spring/charge model uses the following coefficients:

$$\begin{aligned}c_{vv} &= \beta L^2 \\c_{ev} &= \beta \gamma_{ev} L^2 \\c_{ee} &= \beta \gamma_{ee} L^2 \\c_{center} &= \beta g_{center} L^2 \\c_{dangling} &= \beta \gamma_{dangling} L^2\end{aligned}$$

The Typst parameter names are beta for β , gamma-ev for γ_{ev} , gamma-ee for γ_{ee} , g-center for g_{center} , and gamma-dangling for $\gamma_{dangling}$. The spring stiffness k is k-spring, and the softening constant ε is eps.

In layout-algo: "anneal", linnest minimizes an energy:

$$\begin{aligned}E &= \sum_{i < j} \frac{1}{2} \frac{c_{vv}}{d(v_i, v_j) + \varepsilon} + \sum_{i, e} \frac{c_{ev}}{d(v_i, e) + \varepsilon} + \sum_{(v, e) \text{ incident}} \frac{1}{2} k (\ell_e - d(v, e))^2 \\&+ \sum_{\text{local edge pairs}} \frac{1}{2} \frac{c_{ee}}{d(e_i, e_j) + \varepsilon} + \sum_{\text{dangling pairs}} \frac{1}{2} \frac{c_{dangling}}{d(e_i, e_j) + \varepsilon} + \sum_i \frac{1}{2} \frac{c_{center}}{d(v_i, 0) + \varepsilon} + p_{\text{cross}} N_{\text{cross}}\end{aligned}$$

.

Here p_{cross} is crossing-penalty and N_{cross} is the number of detected edge crossings. temp, step, seed, steps, epochs, cool, accept-floor, step-shrink, and incremental-energy belong to this simulated annealing mode. crossing-penalty is also anneal-only; force mode does not currently add a crossing force.

In layout-algo: "force", linnest applies the direct forces corresponding to the same vertex-vertex, edge-vertex, incidence spring, local edge-edge, dangling-edge, and center terms. step is the integration step, delta clamps per-step movement, steps and epochs set the iteration budget, cool shrinks the step after each epoch, and early-tol stops when movement is small. z-spring and z-spring-growth are force-only helpers: the integrator gives points temporary z coordinates to break overlaps and pulls them back toward the 2D plane.

directional-force is applied in both modes as an extra bias derived from pin/port direction constraints.

After either graph layout mode, labels are relaxed separately. If L_l is the label target distance and q_l is the label repulsion strength, then $L_l = \alpha_l L$ and $q_l = \beta_l L^2$. The Typst names are label-length-scale for α_l and label-charge for β_l . label-spring is the spring constant pulling each label toward its target. label-steps, label-step, label-early-tol, and label-max-delta-scale control the label relaxation iteration.

Subgraphs

Subgraph objects are opaque zero-copy values.

- `subgraph.label(g, label)` constructs a subgraph from a base62 label.
- `subgraph.bits(g, bits)` constructs a subgraph from a boolean hedge array.
- `subgraph.compass(g, compass)` selects half edges with a DOT compass point.

- `subgraph.to-label(sg)` returns the base62 label.
- `subgraph.hedges(sg)` returns included hedge indices.
- `subgraph.contains(sg, hedge)` tests hedge membership.

Generated Reference

draw

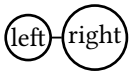
- `draw()`

draw

Draw a graph object with CeTZ.

The graph must already have positions, either from `layout` or from explicit `pos` values passed to `graph.node`, `graph.edge`, or `graph.build`. Paired edges without an explicit edge position use the midpoint of their endpoint nodes. Node and edge Typst style dictionaries or callbacks are evaluated and forwarded to CeTZ.

```
#let positioned = graph.build(
  name: "positioned",
  nodes: (
    (name: "left", pos: graph.pos(x: 0, y: 0)),
    (name: "right", pos: graph.pos(ref: 0, dx: 2.5, dy: 0)),
  ),
  edges: (
    (source: (node: 0), sink: (node: 1)),
    (source: (node: 1), sink: none, pos: graph.pos(ref: 1, dx: 0.9, dy: 0.7)),
  ),
)
#draw(positioned, source-style: (stroke: black + 0.7pt), sink-style: (stroke: black + 0.7pt))
```

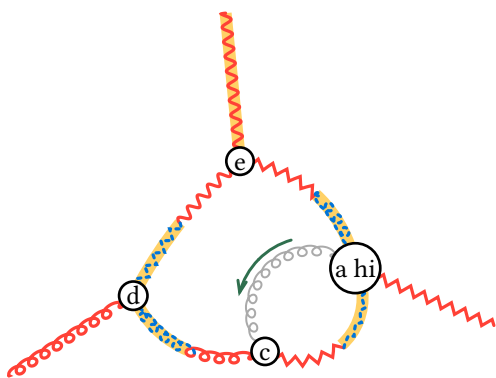


```
#let b = graph.builder(name: "demo")
#let parallel-base-edge = 0
#let source-patterns = (
  "coil",
  "zigzag",
  "coil",
  "wave",
  "zigzag",
  "coil",
  "wave",
  "zigzag",
)
#let sink-patterns = (
  "coil",
  "zigzag",
  "coil",
  "zigzag",
  "coil",
  "wave",
  "coil",
  "wave",
)
#let parallel-layer(edge, mark: none) = if edge.eid == parallel-base-edge {
  (
    offset: -0.5,
    length: 1.5,
    ratio: 0.5,
    resolve-length: "min",
    stroke: (paint: rgb("#2f6f4e"), thickness: 1.1pt, cap: "round"),
    mark: mark,
  )
}
```

```

)
} else { none }
#let stack(base, layer) = if layer == none { base } else { (base, layer) }
#let source-stroke(edge) = if edge.eid == parallel-base-edge {
  (paint: gray, thickness: 0.75pt, cap: "round")
} else {
  (paint: red, thickness: 1.2pt, cap: "round")
}
#let sink-stroke(edge) = if edge.eid == parallel-base-edge {
  (paint: gray, thickness: 0.75pt, cap: "round")
} else {
  (paint: blue, thickness: 1.2pt, cap: "round", dash: "dotted")
}
#let source-style(edge) = {
  let base = (
    stroke: source-stroke(edge),
    pattern: source-patterns.at(edge.eid),
    pattern-amplitude: 0.18,
    pattern-wavelength: 0.55,
    pattern-coil-longitudinal-scale: 1.6,
  )
  stack(base, parallel-layer(edge))
}
#let sink-style(edge) = {
  let base = (
    stroke: sink-stroke(edge),
    pattern: sink-patterns.at(edge.eid),
    pattern-amplitude: 0.18,
    pattern-wavelength: 0.55,
    pattern-coil-longitudinal-scale: 1.6,
  )
  stack(base, parallel-layer(edge, mark: (end: (symbol: "straight"), scale: 0.75)))
}
#let (node: a, builder: b) = graph.node(b, name: "a hi")
#let (node: c, builder: b) = graph.node(b, name: "c")
#let (node: d, builder: b) = graph.node(b, name: "d")
#let (node: e, builder: b) = graph.node(b, name: "e")
#let b = graph.edge(b, source: (node: a), sink: (node: c), pos: graph.pos(x: 0, y: 0.75, mode: "pin"))
#let b = graph.edge(b, source: (node: c), sink: (node: a, compass: "e"))
#let b = graph.edge(b, source: (node: c), sink: (node: d, compass: "e"))
#let b = graph.edge(b, source: (node: e), sink: (node: d, compass: "e"))
#let b = graph.edge(b, source: (node: e), sink: (node: a, compass: "e"))
#let b = graph.edge(b, source: (node: d), sink: none)
#let b = graph.edge(b, source: (node: e, compass: "e"), sink: none)
#let b = graph.edge(b, source: (node: a), sink: none)
#let layed-out = layout(graph.finish(b))
#let east = subgraph.compass(layed-out, "e")
#draw(layed-out, subgraph: east, source-style: source-style, sink-style: sink-style)

```



```

#let p = graph.builder(name: "parallel demo")

```

```

#let (node: pa, builder: p) = graph.node(p, name: "a", pos: graph.pos(y: 0, mode: "pin"))
#let (node: pc, builder: p) = graph.node(p, name: "c", pos: graph.pos(y: 0, mode: "pin"))
#let (node: pc1, builder: p) = graph.node(p, name: "c", pos: graph.pos(y: 0, mode: "pin"))
#let (node: pc2, builder: p) = graph.node(p, name: "c", pos: graph.pos(y: 0, mode: "pin"))
#let p = graph.edge(p, source: (node: pa), sink: (node: pc))
#let p = graph.edge(p, source: (node: pc2), sink: (node: pc1))
#let parallel-edge-style(edge) = (
  offset: -0.5,
  length: 1.4,
  ratio: 0.5,
  resolve-length: "min",
  stroke: (paint: rgb("#2f6f4e"), thickness: 1.1pt, cap: "round"),
)

#let focused-base-style(edge) = (
  stroke: (paint: gray, thickness: 0.7pt, cap: "round"),
  pattern: "coil",
  pattern-amplitude: 0.14,
  pattern-wavelength: 0.45,
  pattern-coil-longitudinal-scale: 1.5,
)

#let focused-source-style(edge) = (focused-base-style(edge), parallel-edge-style(edge))
#let focused-sink-style(edge) = (focused-base-style(edge), parallel-edge-style(edge) + (
  mark: (end: ">"),
))

#draw(layout(graph.finish(p), g-center: 0.004, label-steps: 0,), unit: 1.25, source-style: focused-
source-style, sink-style: focused-sink-style)

```



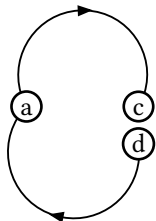
```

#let b = graph.builder(name: "oriented marks")
#let (node: a, builder: b) = graph.node(b, name: "a", pos: graph.pos(x: 0, y: 0, mode: "pin"))
#let (node: c, builder: b) = graph.node(b, name: "c", pos: graph.pos(x: 3, y: 0, mode: "pin"))
#let (node: d, builder: b) = graph.node(b, name: "d", pos: graph.pos(x: 3, y: -1, mode: "pin"))
#let b = graph.edge(b, source: (node: a), sink: (node: c), orientation: "default")
#let b = graph.edge(b, source: (node: a), sink: (node: d), orientation: "reversed")
#let arrow = (
  end: (symbol: ">", fill: black, anchor: "center", shorten-to: auto),
  scale: 0.75,
)

#let oriented-arrow = (
  stroke: black + 0.7pt,
  mark: arrow,
  mark-position: "center-if-dangling",
  mark-orientation: "edge",
)

#draw(layout(graph.finish(b)), unit: 1.4, source-style: oriented-arrow, sink-style: oriented-arrow)

```



Parameters

```
draw(  
  graph: bytes ,  
  scope: dictionary ,  
  unit: int float length ratio ,  
  title: none auto content string ,  
  subgraph: none bytes ,  
  debug: bool int ,  
  node-radius: auto int float array ,  
  node-min-radius: int float ,  
  node-label-padding: int float ,  
  node-fill: any ,  
  node-stroke: any ,  
  node-outset: auto int float ,  
  node-label-style: dictionary ,  
  node-style: dictionary function none ,  
  node-label: auto content string function none ,  
  edge-stroke: any ,  
  edge-offset: int float ,  
  edge-length: none int float ,  
  edge-ratio: none int float ,  
  edge-resolve-length: string function ,  
  edge-accuracy: float ,  
  edge-optimize: bool ,  
  source-style: dictionary array function none ,  
  sink-style: dictionary array function none ,  
  edge-label: content string function none ,  
  edge-omega: float ,  
  edge-trim-accuracy: float ,  
  padding: none int float array dictionary ,  
  debug-edge-radius: int float ,  
  debug-edge-fill: any ,  
  debug-edge-stroke: any ,  
  debug-edge-label-fill: any ,  
  subgraph-edge-style: dictionary ,  
  subgraph-edge-underlay: bool  
) -> content
```

graph bytes

Graph object with positions from layout or explicit graph API pos fields.

scope dictionary

Additional values merged into node and edge callback dictionaries.

Default: (:)

unit int or float or length or ratio

Coordinate length for one graph-layout unit. Numbers are interpreted as em.

Default: 1

title `none` or `auto` or `content` or `string`

Optional title displayed above the diagram. Use auto for the graph name.

Default: `none`

subgraph `none` or `bytes`

Optional subgraph whose half-edges are shaded.

Default: `none`

debug `bool` or `int`

Debug level. 1 enables CeTZ canvas debug; 2 also marks edge positions.

Default: `false`

node-radius `auto` or `int` or `float` or `array`

Default CeTZ node radius. Use auto to fit the node label.

Default: `auto`

node-min-radius `int` or `float`

Minimum radius used when node-radius is auto.

Default: `0.16`

node-label-padding `int` or `float`

Extra canvas-unit padding around labels when node-radius is auto.

Default: `0.08`

node-fill `any`

Default CeTZ node fill.

Default: `white`

node-stroke `any`

Default CeTZ node stroke.

Default: `black`

node-outset `auto` or `int` or `float`

Edge clearance from node centers. auto uses each node circle radius. Increase this when labels extend beyond the circle.

Default: `auto`

node-label-style dictionary

Default node-label style forwarded to `cetz.draw.content`.

Default: `(:)`

node-style dictionary or function or none

Node style dictionary or callback. A callback receives node data.

Default: `(:)`

node-label auto or content or string or function or none

Node label content or callback. auto uses the node name.

Default: `auto`

edge-stroke any

Default CeTZ edge stroke.

Default: `0.1em`

edge-offset int or float

Default normal offset for edge paths. Applied to the base edge geometry before patterns; node outsets then trim the shifted path.

Default: `_edge-geometry-defaults.offset`

edge-length none or int or float

Maximum visible arc length for centered parallel edge paths. none keeps the full shifted path.

Default: `_edge-geometry-defaults.length`

edge-ratio none or int or float

Maximum visible fraction of the base edge length for centered parallel edge paths. Combined with edge-length according to edge-resolve-length.

Default: `_edge-geometry-defaults.ratio`

edge-resolve-length string or function

Resolve edge-length and edge-ratio. Accepted string values are "min"/"shorter", "max"/"longer", "length"/"fixed", "ratio"/"relative", or "none"/"full". A function receives (base-length, length, ratio).

Default: `_edge-geometry-defaults.resolve-length`

edge-accuracy float

Arc-length accuracy for fitted parallel edge paths.

Default: `_edge-geometry-defaults.accuracy`

edge-optimize bool

Let Kurbo optimize the fitted parallel path.

Default: `_edge-geometry-defaults.optimize`

source-style dictionary or array or function or none

Source half-edge style dictionary, array of layer dictionaries, or callback. `mark-position`: "center-if-dangling" keeps an end marker at the paired-edge split point while centering it on dangling half edges. `mark-orientation`: "edge" makes a mark follow `edge.orientation` instead of raw path direction; reversed edges move the mark to the sink half and flip it, while undirected edges suppress it.

Default: `(:)`

sink-style dictionary or array or function or none

Sink half-edge style dictionary, array of layer dictionaries, or callback. A callback receives edge data. `mark-position`: "center-if-dangling" has the same dangling-edge behavior as for `source-style`, and `mark-orientation`: "edge" participates in the same orientation-aware mark placement.

Default: `(:)`

edge-label content or string or function or none

Edge label content or callback. A callback receives edge data.

Default: `none`

edge-omega float

Hobby curl used at the endpoints of paired edge curves.

Default: `1.0`

edge-trim-accuracy float

Arc-length accuracy for trimming edge curves at node outsets.

Default: `0.001`

padding none or int or float or array or dictionary

CeTZ canvas padding.

Default: `0.4`

debug-edge-radius `int` or `float`

Radius for edge-position markers shown at `debug >= 2`.

Default: `0.08`

debug-edge-fill `any`

Fill for edge-position markers shown at `debug >= 2`.

Default: `rgb("#ff9f1c")`

debug-edge-stroke `any`

Stroke for edge-position markers shown at `debug >= 2`.

Default: `rgb("#d72638") + 0.35pt`

debug-edge-label-fill `any`

Label fill for edge-position markers shown at `debug >= 2`.

Default: `rgb("#7a1020")`

subgraph-edge-style `dictionary`

CeTZ style used to shade half-edges included in subgraph.

Default: `(stroke: rgb("#ffd166") + 4.5pt)`

subgraph-edge-underlay `bool`

Draw subgraph shading below the normal half-edge style.

Default: `true`

- `layout()`

Variables

- `graph`
- `curve`
- `physics`
- `subgraph`

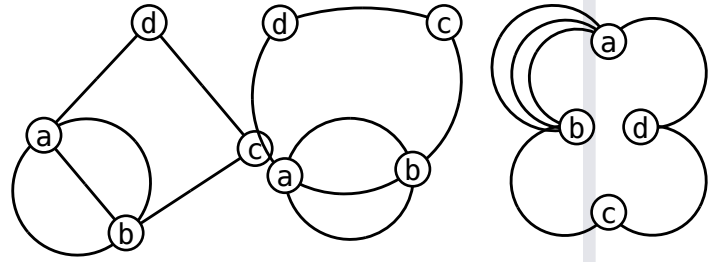
layout

Apply the linnest layout pass to a graph object.

This is intentionally a second step: construct or parse a graph first, then call `layout`. Set `layout-algo` to "force" for deterministic force integration, "anneal" for simulated annealing, "tree" for a traversal tree placement, or "dot" for a layered directed placement.

```
#let g = graph.parse("digraph partial { a:s -> b:s;a -> b;a -> b; b:s -> c:s; c:s -> d:s; d:s -> a:s }").at(0)
#let south = subgraph.compass(g,"s")
#let gf = layout(layout(g, layout-algo: "force"), layout-algo: "tree", layout-nodes: "fixed",subgraph:south)
#let ga = layout(g, layout-algo: "anneal")
```

```
#let gt = layout(layout(g, layout-algo:
"tree", layout-roots: (2)), layout-algo:
"anneal", layout-nodes: "fixed", gamma-
ee:0.1, gamma-ev:1.5, beta:20, length-scale:0.4)
#grid(columns: 3, gutter: 2cm, draw(gf),
draw(ga), draw(gt))
```



```
#let g = graph.parse("digraph partial { a -> b;
b -> c; c -> d; d -> a }").at(0)
#let tree = graph.forests(g).at(0)
#let g = layout(g, layout-algo: "tree",
subgraph: tree)
#graph.edges(g).map(edge => edge.pos)
```

```
(
(x: 0.0, y: 1.5),
(x: 0.0, y: 4.5),
(x: 0.0, y: 7.5),
(x: 0.0, y: 4.5),
)
```

Parameters

```
layout(  
  graph: bytes ,  
  subgraph: none bytes ,  
  viewport-w: float ,  
  viewport-h: float ,  
  tree-dx: float ,  
  tree-dy: float ,  
  steps: int ,  
  seed: int ,  
  step: float ,  
  step-shrink: float ,  
  cool: float ,  
  accept-floor: float ,  
  early-tol: float ,  
  temp: float ,  
  delta: float ,  
  beta: float ,  
  k-spring: float ,  
  g-center: float ,  
  epochs: int ,  
  crossing-penalty: float ,  
  gamma-dangling: float ,  
  gamma-ee: float ,  
  directional-force: float ,  
  label-length-scale: float ,  
  label-spring: float ,  
  label-charge: float ,  
  label-steps: int ,  
  label-step: float ,  
  label-early-tol: float ,  
  label-max-delta-scale: float ,  
  gamma-ev: float ,  
  eps: float ,  
  incremental-energy: bool ,  
  layout-algo: string ,  
  layout-nodes: string ,  
  layout-roots: array ,  
  z-spring: float ,  
  z-spring-growth: float ,  
  length-scale: float  
) -> bytes
```

graph bytes

Graph object returned by `graph.build`, `graph.finish`, or `graph.parse`.

subgraph none or bytes

Optional subgraph object to lay out. With "tree" and "dot", other edges are drawn from the resulting node positions as straight lines. With "force" and "anneal", nodes and edges outside the subgraph are fixed boundary points during optimization.

Default: none

viewport-w float

Width of the layout viewport used to derive the natural spring length. Applies to both "force" and "anneal".

Default: 10.0

viewport-h float

Height of the layout viewport used to derive the natural spring length. Applies to both "force" and "anneal".

Default: 10.0

tree-dx float

Horizontal spacing multiplier for the initial traversal-tree placement. Applies before both "force" and "anneal".

Default: 0.9

tree-dy float

Vertical spacing multiplier for the initial traversal-tree placement. Applies before both "force" and "anneal".

Default: 1.2

steps int

Iterations per epoch. In "force" mode this is the number of force integration steps; in "anneal" mode this is the number of proposals per temperature epoch.

Default: `int(sys.inputs.at("steps", default: "30"))`

seed int

Seed for deterministic initialization, force-mode jitter, and annealing proposals. Applies to both modes.

Default: `int(sys.inputs.at("seed", default: "2"))`

step float

Initial movement scale. "force" multiplies computed forces by this value; "anneal" uses it as the proposal step size.

Default: 0.81

step-shrink float

Anneal-only step shrink factor, applied when an epoch's acceptance ratio falls below `accept-floor`.

Default: 0.21

cool float

Cooling factor applied once per epoch. "anneal" cools temp; "force" shrinks step.

Default: 0.85

accept-floor float

Anneal-only acceptance-ratio threshold below which step is shrunk by step-shrink.

Default: 0.15

early-tol float

Force-only early stop threshold for maximum movement in one step. The annealing schedule stores this value but does not currently use it for stopping.

Default: 1e-6

temp float

Anneal-only initial temperature used in the Metropolis acceptance test.

Default: 0.3

delta float

Force-mode maximum movement clamp per point and per step.

Default: 0.4

beta float

Base repulsion strength for vertex-vertex interactions. Also scales gamma-ev, gamma-ee, gamma-dangling, and g-center. Applies to both modes through the shared spring energy.

Default: 50.0

k-spring float

Spring stiffness for node-to-edge incidence lengths. Applies to both modes.

Default: 11.0

g-center float

Centering strength relative to beta. Applies to both modes.

Default: 0.005

epochs int

Number of epochs. Both modes run up to steps iterations inside each epoch.

Default: 30

crossing-penalty float

Anneal-only fixed energy penalty per detected edge crossing. The direct force integrator does not currently add a crossing force.

Default: 30.0

gamma-dangling float

Repulsion for dangling half edges, relative to beta. Applies to both modes through the shared spring energy.

Default: 5.0

gamma-ee float

Local edge-edge repulsion, relative to beta. Applies to both modes.

Default: 0.1

directional-force float

Bias that pushes points in directions implied by pin/port constraints. Applies to both modes.

Default: 5.0

label-length-scale float

Edge-label target offset as a multiple of the graph spring length. Label layout runs after both graph layout modes.

Default: 0.6

label-spring float

Spring strength pulling each label toward its target offset. Label layout runs after both modes.

Default: 23.0

label-charge float

Repulsion strength between labels and graph points, scaled by spring length squared. Label layout runs after both modes.

Default: 3.0

label-steps int

Maximum number of post-layout label relaxation steps. Set to 0 to skip label placement. Applies after both modes.

Default: 20

label-step float

Label relaxation step size. Applies after both modes.

Default: 0.15

label-early-tol float

Label relaxation early stop threshold. Applies after both modes.

Default: 1e-3

label-max-delta-scale float

Label movement clamp as a multiple of spring length. Applies after both modes.

Default: 0.5

gamma-ev float

Edge-vertex repulsion, relative to beta. Applies to both modes.

Default: 0.01

eps float

Softening epsilon used in inverse-square force/energy terms. Applies to both modes.

Default: 1e-4

incremental-energy bool

Whether annealing updates cached energy by local deltas. This is anneal-only; force mode computes direct forces instead of energies.

Default: true

layout-algo string

Layout algorithm. Use "force" for direct force integration, "anneal" for simulated annealing against the spring energy, "tree" for a traversal-tree placement, or "dot" for a layered directed placement.

Default: "force"

layout-nodes string

Node movement policy. "layout" lets the layout algorithm move nodes. "fixed" keeps every node at its current position for this layout pass and only moves edge control points. With subgraph, only edges in the subgraph are moved; other edge control points stay at their current positions.

Default: "layout"

layout-roots array

Ordered node indices used as preferred roots for "tree" and "dot". Roots outside the selected node set are ignored. Remaining components are laid out afterward in graph order.

Default: ()

z-spring float

Force-only spring pulling temporary z coordinates back toward the layout plane. Use with z-spring-growth to help separate overlapping points during integration.

Default: 0.05

z-spring-growth float

Force-only per-epoch multiplier for z-spring.

Default: **1.3**

length-scale float

Natural spring-length multiplier. This scales the graph's preferred edge length and the repulsive coefficients derived from it. Applies to both modes.

Default: **0.5**

graph module

Graph namespace.

Graph objects are opaque zero-copy values. Keep them opaque and pass them back to this module, `subgraph`, or `layout()`.

curve module

Curve namespace.

Helpers for splitting and drawing Bezier edge geometry.

physics module

Physics edge-style namespace.

Reusable particle-line styles and callbacks for diagrams drawn with `draw`.

subgraph module

Subgraph namespace.

Subgraph objects are opaque zero-copy values. They can be passed to `graph.nodes` and `graph.edges`.

graph

- `build()`
- `builder()`
- `cycles()`
- `dot()`
- `edge()`
- `edges()`
- `finish()`
- `forests()`
- `group()`
- `info()`
- `join()`
- `node()`

- nodes()
- parse()
- pos()

build

Build one graph object from node and edge arrays.

This is convenience sugar over `builder()`, `node()`, `edge()`, and `finish()`.

```
#let g = graph.build(
  name: "demo",
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1))),),
)
#graph.info(g).name
```

demo

Parameters

```
build(
  name: none string ,
  statements: dictionary ,
  edge-statements: dictionary ,
  source-style-eval: none string ,
  sink-style-eval: none string ,
  label-eval: none string ,
  node-statements: dictionary ,
  nodes: array ,
  edges: array
) -> bytes
```

name none or string

Graph name.

Default: none

statements dictionary

Graph-level DOT statements.

Default: (:)

edge-statements dictionary

Default DOT statements for edges. String values may use {name} placeholders that expand from each edge's merged statement dictionary.

Default: (:)

source-style-eval none or string

Default always-evaluated Typst style for source half-edges. This is merged into edge-statements as source-style-eval.

Default: none

sink-style-eval `none` or `string`

Default always-evaluated Typst style for sink half-edges. This is merged into edge-statements as sink-style-eval.

Default: `none`

label-eval `none` or `string`

Default always-evaluated Typst edge label template. This is merged into edge-statements as label-eval.

Default: `none`

node-statements `dictionary`

Default DOT statements for nodes.

Default: `(:)`

nodes `array`

Node specifications. Each node may define name, index, pos, shift, and statements. This matches the per-node parameters accepted by `node()`.

Default: `()`

edges `array`

Edge specifications. Each edge may define source, sink, orientation, flow, id, pos, shift, label-pos, label-angle, bend, statements, source-style-eval, sink-style-eval, and label-eval. The source and sink fields use the same half-edge dictionaries accepted by `edge()`.

Default: `()`

builder

Create a builder object.

statements, node-statements, and edge-statements set graph-level DOT attributes that are carried into the finished graph. String values may use `{name}` placeholders; edge defaults expand after per-edge statements are merged, so an edge-statements, source-style-eval, or sink-style-eval value can refer to a label supplied by `edge()`.

```
#let b = graph.builder(  
  name: "demo",  
  statements: (full_num: "x + y"),  
  edge-statements: (display-label: "{label}"),  
  source-style-eval: "(stroke: red + 0.5pt)",  
  sink-style-eval: "(stroke: blue + 0.5pt)",  
)  
#let (node: a, builder: b) = graph.node(b, name:  
  "a")  
#a
```

0

Parameters

```
builder(  
  name,  
  statements,  
  node-statements,  
  edge-statements,  
  source-style-eval,  
  sink-style-eval,  
  label-eval  
) -> bytes
```

cycles

Return subgraph objects for the graph's cycle basis.

```
#let g = graph.build(  
  nodes: ((name: "a"), (name: "b")),  
  edges: ((source: (node: 0), sink: (node:  
1))),),  
)  
#graph.cycles(g).len()
```

0

Parameters

```
cycles(graph) -> array
```

dot

Serialize a graph object to DOT.

```
#let g = graph.build(  
  name: "demo",  
  nodes: ((name: "a"), (name: "b")),  
  edges: ((source: (node: 0), sink: (node:  
1))),),  
)  
#graph.dot(g).contains("digraph demo")
```

true

Parameters

```
dot(graph) -> string
```

edge

Add a paired or external edge to a builder object.

Set source to none or sink to none for an external edge. Half-edge dictionaries accept node, statement, id, port-label, compass, and in-subgraph.

```
#let b = graph.builder(name: "demo")  
#let (node: a, builder: b) = graph.node(b, name:  
"a")  
#let (node: c, builder: b) = graph.node(b, name:  
"c")  
#let b = graph.edge(  
  b,  
  source: (node: a, compass: "e"),  
  sink: (node: c, compass: "w"),
```

```
statements: (label: "a-c"),
)
#graph.edges(graph.finish(b)).len()
```

1

Parameters

```
edge(
  builder,
  source,
  sink,
  orientation,
  flow,
  id,
  pos,
  shift,
  label-pos,
  label-angle,
  bend,
  statements,
  source-style-eval,
  sink-style-eval,
  label-eval
) -> bytes
```

edges

Return edge records, optionally filtered by an subgraph object.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0, compass: "e"),
  sink: (node: 1))),
)
#let east = subgraph.compass(g, "e")
#graph.edges(g, subgraph: east).len()
```

1

Parameters

```
edges(
  graph,
  subgraph
) -> array
```

finish

Finish a builder object into a graph object.

```
#let b = graph.builder(name: "demo")
#let (node: a, builder: b) = graph.node(b, name:
"a")
```

```
#let g = graph.finish(b)
#graph.info(g).name
```

demo

Parameters

`finish(builder)` -> `bytes`

forests

Return subgraph objects for the graph's spanning forests.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1)),),
)
#graph.forests(g).len()
```

1

Parameters

`forests(graph)` -> `array`

group

Create a grouped placement coordinate.

side: "+" keeps the solved coordinate non-negative and side: "-" keeps it non-positive. Groups are layout constraints and therefore require pin placement, which is the `graph.pos` default.

```
#graph.group("right", side: "+")
```

(kind: "group", name: "right", side: "+")

Parameters

```
group(
  name,
  side
) -> dictionary
```

info

Return graph metadata.

The result has name, global-statements, edge-statements, and node-statements.

```
#let g = graph.build(name: "demo", nodes:
((name: "a"),))
#graph.info(g).name
```

demo

Parameters

`info(graph)` -> `dictionary`

join

Join two graphs by matching dangling half-edge data on key.

Supported key values are "statement", "port-label", "compass", and "id".

```
#let left = graph.build(  
  nodes: ((name: "a"),),  
  edges: ((source: none, sink: (node: 0,  
    statement: "j")),),  
)  
#let right = graph.build(  
  nodes: ((name: "b"),),  
  edges: ((source: (node: 0, statement: "j"),  
    sink: none),),  
)  
#graph.edges(graph.join(left, right, key:  
  "statement")).len()
```

1

Parameters

```
join(  
  left,  
  right,  
  key  
) -> bytes
```

node

Add a node to a builder object.

Returns a dictionary with fields node and builder, so callers can write #let (node: a, builder: b) = graph.node(b, name: "a"). The returned builder is the value to pass to the next node(), edge(), or finish() call.

```
#let b = graph.builder(name: "demo")  
#let (node: a, builder: b) = graph.node(b, name:  
  "a")  
#let (node: c, builder: b) = graph.node(b, name:  
  "c")  
#repr((a, c))
```

(0, 1)

Parameters

```
node(  
  builder,  
  name,  
  index,  
  pos,  
  shift,  
  statements  
) -> dictionary
```

nodes

Return node records, optionally filtered by an subgraph object.

```
#let g = graph.build(nodes: ((name: "a"), (name:  
  "b")))  
#graph.nodes(g).map(node => node.name).join(",  
  ")
```

a, b

Parameters

```
nodes(  
  graph,  
  subgraph  
) -> array
```

parse

Parse one or more DOT digraphs into graph objects.

```
#let graphs = graph.parse("digraph first { a ->  
b }")  
#graphs.len()
```

1

Parameters

```
parse(input) -> array
```

pos

Create a first-class graph placement.

The default mode: "pin" turns numeric and grouped coordinates into layout constraints and also makes the coordinates immediately drawable without a layout pass. Use mode: "start" when the coordinate should only seed the layout. ref may reference a previously created node index and combines with dx and dy.

```
#graph.pos(x: 0, y: graph.group("row"), mode:  
"pin")
```

```
(  
  mode: "pin",  
  x: 0,  
  y: (kind: "group", name: "row", side:  
none),  
)
```

Parameters

```
pos(  
  x,  
  y,  
  ref,  
  dx,  
  dy,  
  mode  
) -> dictionary
```

physics

- dangling-half-edge-index()
- edge-entry()
- edge-index()
- edge-label()
- eval-template()
- interpolate-template()
- label-content()
- momentum-value()
- particle-name()
- sink-stroke()
- sink-style()
- source-stroke()

- `source-style()`
- `stroke-style()`
- `style()`
- `style-dict()`
- `text-value()`

Variables

- `massive`
- `massless`
- `dashed`
- `dotted`
- `fermion-arrow-mark`
- `fermion-flow`
- `wave`
- `coil`
- `zigzag`
- `default-edge`
- `default-map`
- `momentum-arrow-defaults`

dangling-half-edge-index

Return the exposed half-edge index for dangling edges only.

Parameters

`dangling-half-edge-index`(edge) -> `none` `any`

edge-entry

Return the style-map entry for an edge.

Parameters

```
edge-entry(
  edge,
  map,
  default
) -> dictionary
```

edge-index

Return the edge index used by optional edge labels. The DOT id statement wins over the renderer-local eid.

Parameters

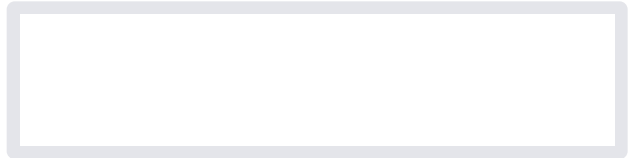
```
edge-index(
  edge,
  fields
) -> none any
```

edge-label

Edge-label callback.

By default this preserves explicit `label-eval`, `display-label`, `label-template`, `label`, and `particle-map` labels. Set any `show-*` option to build a label from selected metadata instead:

```
#let callbacks = physics.style(  
  momentum-arrows: true,  
  show-edge-index: true,  
  show-particle: true,  
)
```



Parameters

```
edge-label(  
  edge,  
  map,  
  default,  
  typst-fields,  
  scope,  
  show-momentum,  
  show-edge-index,  
  show-half-edge-index,  
  show-particle,  
  momentum-fields,  
  edge-index-fields,  
  momentum-prefix,  
  edge-index-prefix,  
  half-edge-index-prefix,  
  particle-prefix,  
  label-separator,  
  ],  
  label-size,  
  label-fill  
) -> none content
```

eval-template

Evaluate a string-valued style template after placeholder interpolation.

Parameters

```
eval-template(  
  template,  
  edge,  
  map,  
  scope  
) -> any
```

interpolate-template

Replace {field} placeholders in a string-valued style template. {{ and }} produce literal braces.

Parameters

```
interpolate-template(  
  template,  
  edge  
) -> none string
```

label-content

Convert a style label value to content. In mode: "eval", string values are evaluated after placeholder interpolation.

Parameters

```
label-content(  
  value,  
  edge,  
  mode,  
  map,  
  scope  
) -> none content
```

momentum-value

Return the momentum field used by optional edge labels.

Parameters

```
momentum-value(  
  edge,  
  fields  
) -> none any
```

particle-name

Return an edge's particle name, stripping the quotes often present in DOT statement values.

Parameters

```
particle-name(edge) -> none string
```

sink-stroke

Sink-half stroke helper. The default lightening makes the two halves encode the graph's source/sink split without requiring arrowheads.

Parameters

```
sink-stroke(  
  c,  
  thickness,  
  dash,  
  lighten  
) -> dictionary
```

sink-style

Style callback for the sink half edge.

`momentum-arrows: true` adds one centered black CeTZ-mark decoration toward the sink node, so momentum arrows always flow from source to sink independently of `edge.orientation`.

Parameters

```
sink-style(  
  edge,  
  map,  
  default,  
  typst-fields,  
  scope,  
  orientation-split,  
  momentum-arrows,  
  momentum-arrow-offset,  
  momentum-arrow-length,  
  momentum-arrow-ratio,  
  momentum-arrow-stroke,  
  momentum-arrow-mark  
) -> dictionary array
```

source-stroke

Source-half stroke helper.

Parameters

```
source-stroke(  
  c,  
  thickness,  
  dash  
) -> dictionary
```

source-style

Style callback for the source half edge.

`momentum-arrows: true` adds one centered black CeTZ-mark decoration while the main edge remains drawn with its normal particle style.

Parameters

```
source-style(  
  edge,  
  map,  
  default,  
  typst-fields,  
  scope,  
  orientation-split,  
  momentum-arrows,  
  momentum-arrow-offset,  
  momentum-arrow-length,  
  momentum-arrow-ratio,  
  momentum-arrow-stroke,  
  momentum-arrow-mark  
) -> dictionary array
```

stroke-style

Build a rounded CeTZ stroke dictionary accepted by `linnest.draw`.

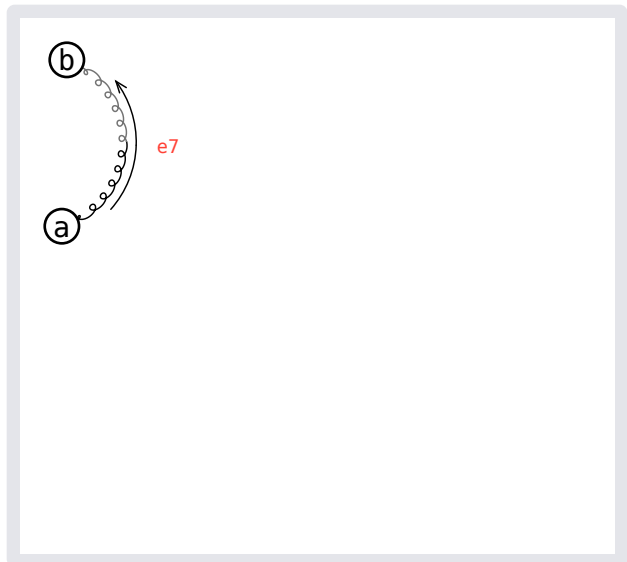
Parameters

```
stroke-style(  
  c,  
  thickness,  
  dash  
) -> dictionary
```

style

Bundle source-style, sink-style, and edge-label callbacks with shared options for `linnest.draw`.

```
#let b = graph.builder(name: "physics")  
#let (node: a, builder: b) = graph.node(b, name: "a")  
#let (node: b-node, builder: b) = graph.node(b, name: "b")  
#let b = graph.edge(  
  b,  
  source: (node: a),  
  sink: (node: b-node),  
  statements: (particle: "g", id: "7"),  
)  
#let callbacks = physics.style(momentum-arrows: true, show-edge-index: true)  
#draw(  
  layout(graph.finish(b)),  
  source-style: callbacks.source-style,  
  sink-style: callbacks.sink-style,  
  edge-label: callbacks.edge-label,  
)
```



Parameters

```
style(  
  map,  
  default,  
  typst-fields,  
  scope,  
  orientation-split,  
  momentum-arrows,  
  momentum-arrow-offset,  
  momentum-arrow-length,  
  momentum-arrow-ratio,  
  momentum-arrow-stroke,  
  momentum-arrow-mark,  
  show-momentum,  
  show-edge-index,  
  show-half-edge-index,  
  show-particle,  
  momentum-fields,  
  edge-index-fields,  
  momentum-prefix,  
  edge-index-prefix,  
  half-edge-index-prefix,  
  particle-prefix,  
  label-separator,  
  ],  
  label-size,  
  label-fill  
) -> dictionary
```

style-dict

Convert a style value to a dictionary. In mode: "eval", string values are evaluated after placeholder interpolation.

Parameters

```
style-dict(  
  value,  
  edge,  
  mode,  
  map,  
  scope  
) -> dictionary
```

text-value

Convert DOT-ish values to plain text.

Parameters

```
text-value(value) -> string
```

massive length

Conventional massive-particle stroke thickness.

massless length

Conventional massless-particle stroke thickness.

dashed array

Dash pattern used for scalar-style lines.

dotted string

Dotted dash style used for ghost-style lines.

fermion-arrow-mark dictionary

CeTZ marker used for fermion particle-flow arrows on the main edge.

fermion-flow dictionary

Mark an edge-map entry as a fermion so the main edge receives one particle-flow arrow that follows edge.orientation. This is separate from optional momentum arrows.

wave dictionary

Photon-style wave pattern.

coil dictionary

Gluon-style coil pattern.

zigzag dictionary

Weak-boson-style zigzag pattern.

default-edge dictionary

Fallback style entry used when an edge has no known particle type.

default-map dictionary

Small built-in particle map for standalone physics diagrams. GammaLoop generated `edge-style.typ` files pass their model-specific map to `style`.

momentum-arrow-defaults dictionary

Default style for source-to-sink momentum arrow layers.

subgraph

- `bits()`
- `compass()`
- `contains()`
- `hedges()`
- `label()`
- `to-label()`

bits

Construct an subgraph object from a boolean hedge array.

```
#let g = graph.build(  
  nodes: ((name: "a"), (name: "b")),  
  edges: ((source: (node: 0), sink: (node:  
1))),,  
)  
#let first = subgraph.bits(g, (true, false))  
#subgraph.contains(first, 0)
```

true

Parameters

```
bits(  
  graph,  
  bits  
) -> bytes
```

compass

Construct an subgraph object from a DOT compass point such as "n" or "s".

```
#let g = graph.build(  
  nodes: ((name: "a"), (name: "b")),  
  edges: ((source: (node: 0, compass: "e"),  
    sink: (node: 1)),),  
)  
#subgraph.hedges(subgraph.compass(g, "e").len())
```

1

Parameters

```
compass(  
  graph,  
  compass  
) -> bytes
```

contains

Test whether an subgraph object includes a hedge.

```
#let g = graph.build(  
  nodes: ((name: "a"), (name: "b")),  
  edges: ((source: (node: 0), sink: (node:  
1))),),  
)  
#subgraph.contains(subgraph.bits(g, (true,  
false)), 0)
```

true

Parameters

```
contains(  
  subgraph,  
  hedge  
) -> bool
```

hedges

Return the hedge indices included in an subgraph object.

```
#let g = graph.build(  
  nodes: ((name: "a"), (name: "b")),  
  edges: ((source: (node: 0), sink: (node:  
1))),),  
)  
#subgraph.hedges(subgraph.bits(g, (true,  
false)))
```

(0,)

Parameters

```
hedges(subgraph) -> array
```

label

Construct an subgraph object from a base62 label.

```
#let g = graph.build(  
  nodes: ((name: "a"), (name: "b")),
```

```
edges: ((source: (node: 0, compass: "e"),
sink: (node: 1))),
)
#let east = subgraph.compass(g, "e")
#let same = subgraph.label(g, subgraph.to-
label(east))
#subgraph.hedges(same).len()
```

1

Parameters

```
label(
  graph,
  label
) -> bytes
```

to-label

Convert an subgraph object to its base62 label.

```
#let g = graph.build(
  nodes: ((name: "a"), (name: "b")),
  edges: ((source: (node: 0), sink: (node:
1))),),
)
#subgraph.to-label(subgraph.bits(g, (true,
false)))
```

1

Parameters

```
to-label(subgraph) -> string
```